Lab 2: Developing Quantum Circuits in Python with Qiskit

Welcome back to our quantum lab series!

In **Lab 1**, we learned to:

- Set up our quantum programming environment.
- Create single-qubit circuits.
- Apply basic Pauli gates.
- Simulate our circuits in multiple ways.

Now, in **Lab 2**, we'll take those fundamentals and start building *more interesting* circuits.

By the end of this lab, you will:

1. Create a **GHZ state** to explore multi-qubit entanglement.
2. Prepare the **|+⟩ state** and measure it in different bases.
3. Implement a **SWAP** operation using only CNOT (CX) gates.
4. Visualize results on both histograms and Bloch spheres.

We'll continue using Qiskit and its AerSimulator so that we don't depend on real quantum hardware (which has noise and queues).

1. Setup and Installation

We need to make sure Qiskit and related libraries are available.
Run this cell **only if** you haven't installed them yet.

```
%pip install qiskit qiskit-aer matplotlib
```

```
Collecting qiskit
  Downloading qiskit-2.6.1-cp310-abi3-win_amd64.whl.metadata (14 kB)
Collecting qiskit-aer
  Downloading qiskit_aer-0.18.2-cp311-cp311-win_amd64.whl.metadata (8.9 kB)
Requirement already satisfied: matplotlib in C:\Users\sonup\AppData\Roaming\Py
Collecting numpy<3,>=2.0 (from qiskit)
  Using cached numpy-2.4.6-cp311-cp311-win_amd64.whl.metadata (6.6 kB)
Requirement already satisfied: scipy>=1.14 in C:\Users\sonup\AppData\Roaming\P
Collecting rustworkx>=0.15.0 (from qiskit)
  Downloading rustworkx-0.18.1-cp310-abi3-win_amd64.whl.metadata (10 kB)
Collecting dill>=0.3 (from qiskit)
  Using cached dill-0.4.1-py3-none-any.whl.metadata (10 kB)
```

```
Collecting stevedore>=3.0.0 (from qiskit)
  Downloading stevedore-5.9.1-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: typing-extensions in C:\Users\sonup\AppData\Roa
Requirement already satisfied: psutil>=5 in C:\Users\sonup\AppData\Roaming\Pyt
Requirement already satisfied: python-dateutil>=2.8.0 in C:\Users\sonup\AppDat
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\sonup\AppData\Roam
Requirement already satisfied: cycler>=0.10 in C:\Users\sonup\AppData\Roaming\
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\sonup\AppData\Roa
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\sonup\AppData\Roa
Requirement already satisfied: packaging>=20.0 in d:\Anaconda3\envs\tfenv\Lib\
Requirement already satisfied: pillow>=9 in C:\Users\sonup\AppData\Roaming\Pyt
Requirement already satisfied: pyparsing>=3 in C:\Users\sonup\AppData\Roaming\
Requirement already satisfied: six>=1.5 in C:\Users\sonup\AppData\Roaming\Pyth
Downloading qiskit-2.6.1-cp310-abi3-win_amd64.whl (9.3 MB)
   ---------------------------------------- 0.0/9.3 MB ? eta -:--:--
   ---------------------------------------  9.2/9.3 MB 47.7 MB/s eta 0:00:01
   ---------------------------------------- 9.3/9.3 MB 38.4 MB/s  0:00:00
Using cached numpy-2.4.6-cp311-cp311-win_amd64.whl (12.6 MB)
Downloading qiskit_aer-0.18.2-cp311-cp311-win_amd64.whl (9.9 MB)
   ---------------------------------------- 0.0/9.9 MB ? eta -:--:--
   ---- ----------------------------------- 1.0/9.9 MB ? eta -:--:--
   ----------- ---------------------------- 2.9/9.9 MB 6.5 MB/s eta 0:00:02
   -------------- ------------------------- 3.7/9.9 MB 5.5 MB/s eta 0:00:02
   ------------------------ --------------- 6.3/9.9 MB 8.8 MB/s eta 0:00:01
   ---------------------------------- ----- 8.4/9.9 MB 8.4 MB/s eta 0:00:01
   ---------------------------------- ----- 8.4/9.9 MB 8.4 MB/s eta 0:00:01
   ---------------------------------- ----- 8.4/9.9 MB 8.4 MB/s eta 0:00:01
   ---------------------------------------  9.7/9.9 MB 5.9 MB/s eta 0:00:01
   ---------------------------------------- 9.9/9.9 MB 5.7 MB/s  0:00:01
Using cached dill-0.4.1-py3-none-any.whl (120 kB)
Downloading rustworkx-0.18.1-cp310-abi3-win_amd64.whl (2.3 MB)
   ---------------------------------------- 0.0/2.3 MB ? eta -:--:--
   ---------------------------------------- 2.3/2.3 MB 18.5 MB/s  0:00:00
Downloading stevedore-5.9.1-py3-none-any.whl (54 kB)
Installing collected packages: stevedore, numpy, dill, rustworkx, qiskit, qisk

   ---------------------------------------- 0/6 [stevedore]
   ---------------------------------------- 0/6 [stevedore]
   ---------------------------------------- 0/6 [stevedore]
   ---------------------------------------- 0/6 [stevedore]
   ---------------------------------------- 0/6 [stevedore]
  Attempting uninstall: numpy
   ---------------------------------------- 0/6 [stevedore]
    Found existing installation: numpy 1.26.4
   ---------------------------------------- 0/6 [stevedore]
```

2. Imports and Configuration

Here we import:

- Essential Qiskit classes for circuits and simulation.
- Visualization tools for histogram plots and Bloch spheres.
- `Statevector` for representing and inspecting quantum states exactly.

- A fixed random seed to make sure our results are reproducible (important for consistent demos).

```
import qiskit
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator, Aer
from qiskit.visualization import plot_histogram
import matplotlib
from math import pi, sqrt #pi = 3.14
from qiskit.visualization import plot_bloch_multivector
from qiskit.visualization import plot_histogram
from IPython.display import display
from qiskit.quantum_info import Statevector
```

3. A Reusable Run-and-Plot Function

Running a circuit is a bit repetitive:

1. **Transpile** — prepare code for the backend.
2. **Simulate** — execute and get results.
3. **Visualize** — show measurement counts.

We'll wrap this in a single helper function so we can focus on quantum thinking, not boilerplate.

```
SEED = 2024
sim = AerSimulator(seed_simulator=SEED)

def run_and_plot(qc, shots=1024, title=None):
    tqc = transpile(qc, sim)
    result = sim.run(tqc, shots=shots).result()
    counts = result.get_counts()
    fig = plot_histogram(counts, title=(title or qc.name or "Results"))
    display(fig)
    return counts
```

4. The GHZ State — Multi-Qubit Entanglement

The GHZ state for 3 qubits is:

$|GHZ\rangle = (|000\rangle + |111\rangle) / \sqrt{2}$

Why it’s special:

- It’s maximally entangled: measuring one qubit immediately "collapses" the others.

- All qubits are *perfectly correlated* — they're either all 0 or all 1.

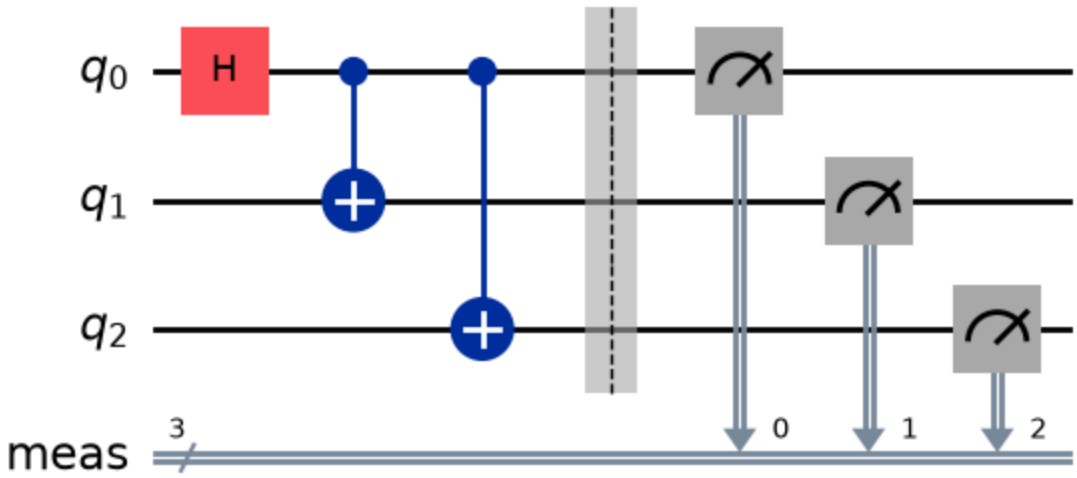
Circuit steps:

1. Apply **H** to qubit 0 → creates 50/50 superposition.
2. Apply **CNOT** from qubit 0 → 1 → entangles them.
3. Apply **CNOT** from qubit 0 → 2 → entangles all three.
4. Measure all qubits.

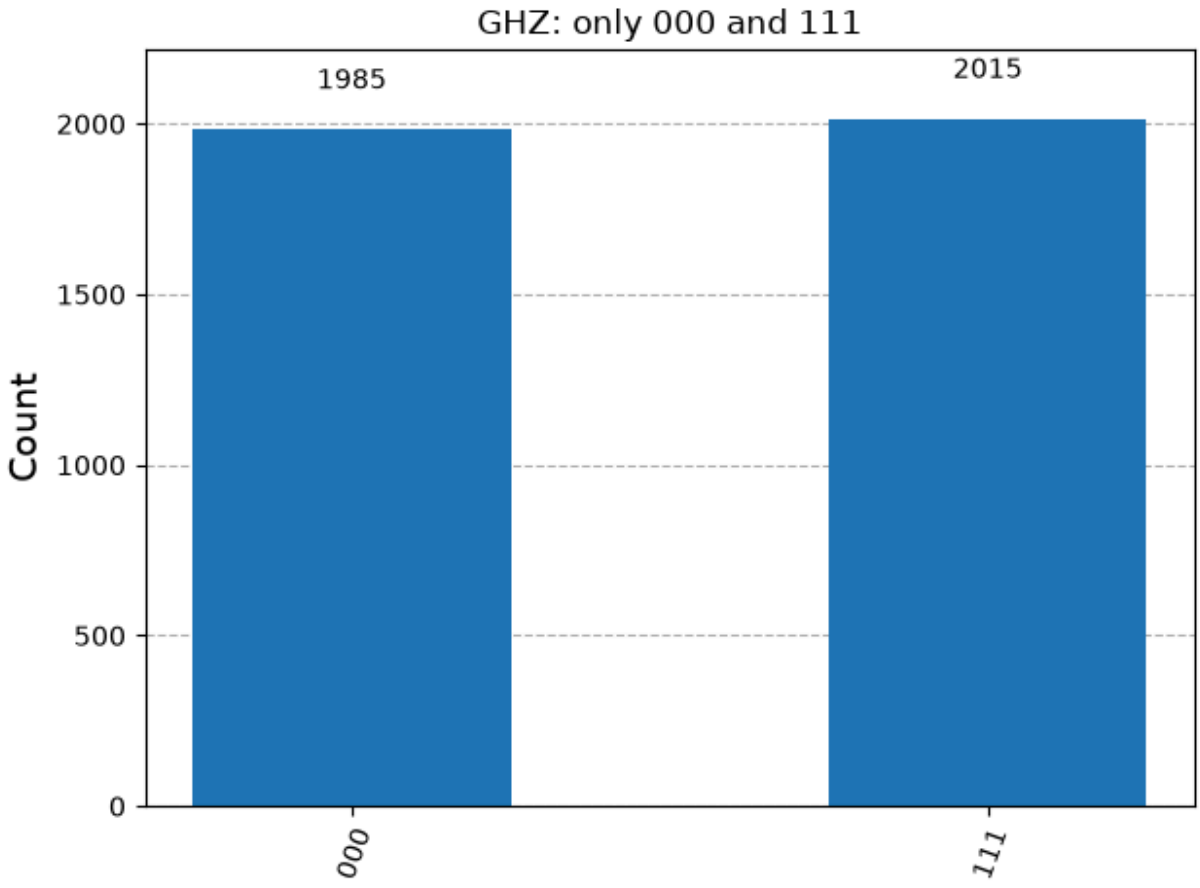
Prediction: In the histogram, only `000` and `111` should appear (≈50% each).

```
qc_ghz = QuantumCircuit(3, name="GHZ")
qc_ghz.h(0)
qc_ghz.cx(0, 1)
qc_ghz.cx(0, 2)
qc_ghz.measure_all()

display(qc_ghz.draw("mpl"))
```




```
counts_ghz = run_and_plot(qc_ghz, shots=4000, title="GHZ: only 000 and 111")
print("Counts:", counts_ghz)
```




```
Counts: {'000': 1985, '111': 2015}
```

5. The |+⟩ State — Measurement Depends on Basis

The + state is:

|+⟩ = (|0⟩ + |1⟩) / √2

We get it by applying **H** to |0⟩.

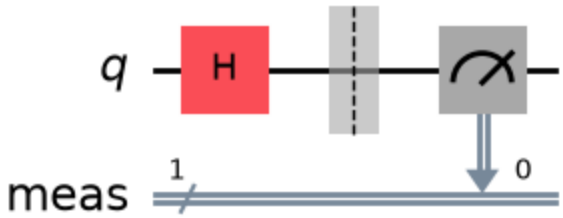
Two measurement strategies:

1. **Z-basis (computational basis)**:
    - No basis change — measure as usual.
    - Expect roughly equal '0' and '1' outcomes.
2. **X-basis**:
    - Apply **H** before measuring to change the basis.
    - Expect always '0', since |+⟩ is the +1 eigenstate of X.

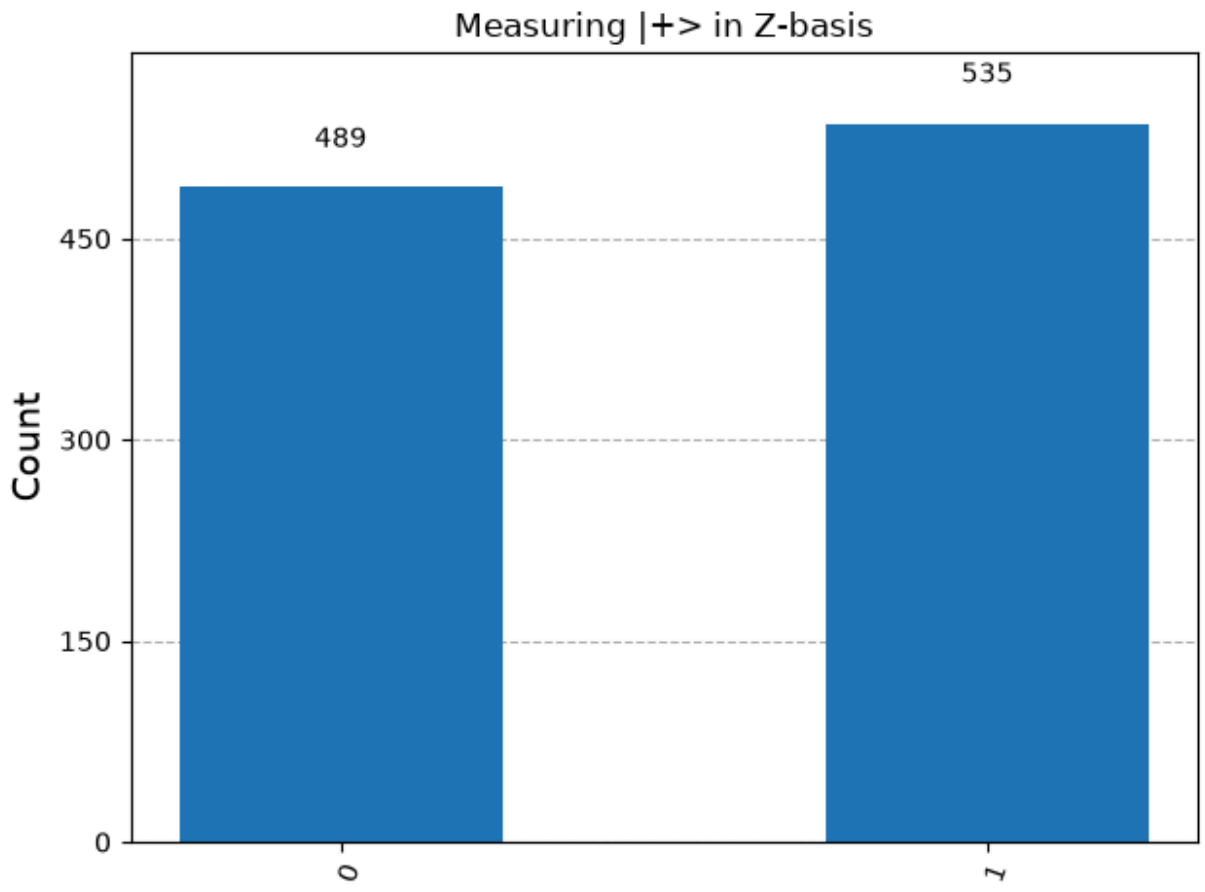
This illustrates a key quantum truth: *what you choose to measure changes what you see*.

```
# Prepare the |+> state
qc_plus_state = QuantumCircuit(1, name="Prepare |+>")
qc_plus_state.h(0)

# Circuit 1: Measure |+> in the Z-basis (should be 50/50)
qc_measure_z = qc_plus_state.copy(name="Measure Z")
qc_measure_z.measure_all()
display(qc_measure_z.draw("mpl"))
```



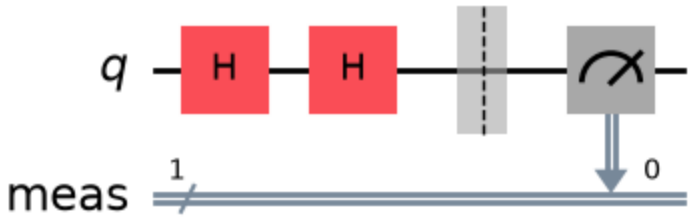

```
run_and_plot(qc_measure_z, title="Measuring |+> in Z-basis")
```



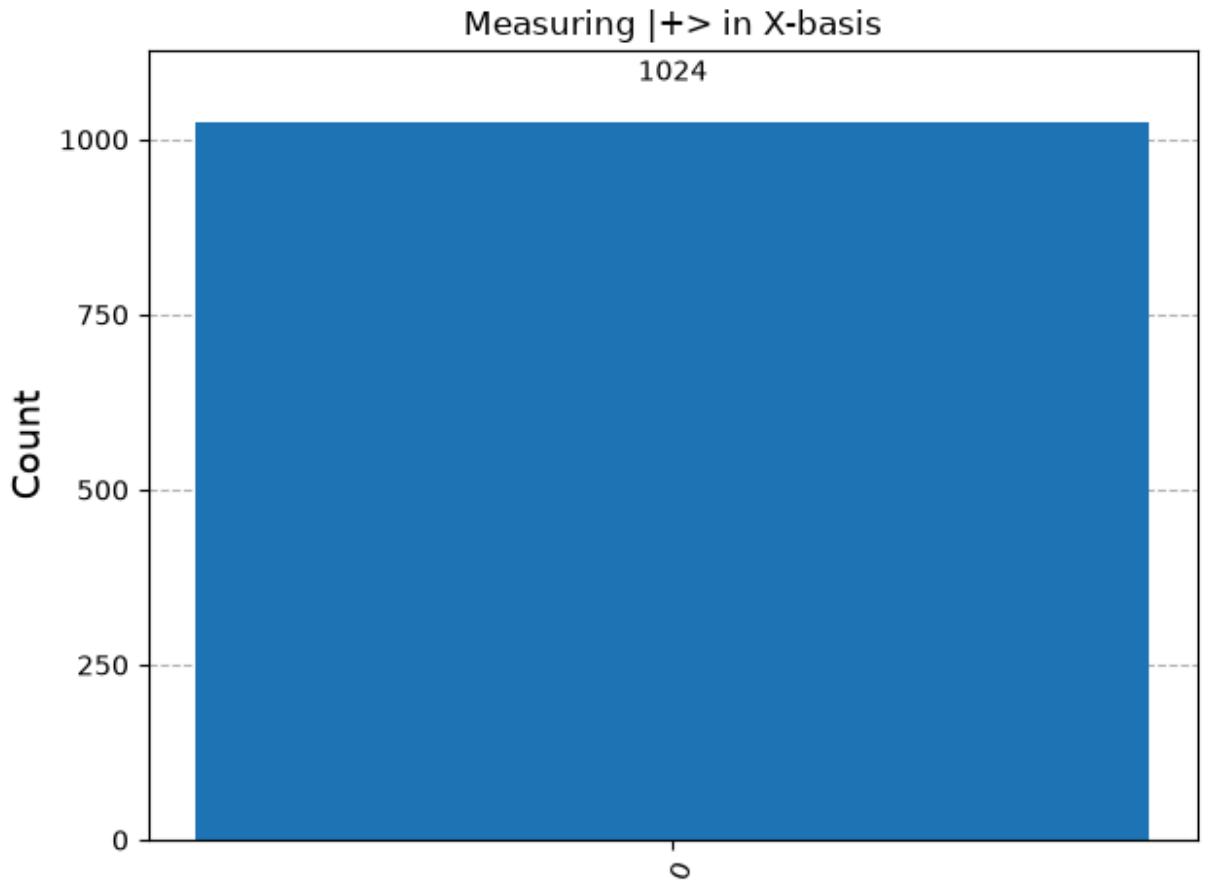

```
{'1': 535, '0': 489}
```

```
# Circuit 2: Measure |+> in the X-basis (should be 100% |+>, so we get "0")
qc_measure_x = qc_plus_state.copy(name="Measure X")
```

```
qc_measure_x.h(0) # Apply H to switch from X-basis to Z-basis
qc_measure_x.measure_all()
display(qc_measure_x.draw("mpl"))
```




```
run_and_plot(qc_measure_x, title="Measuring |+> in X-basis")
```

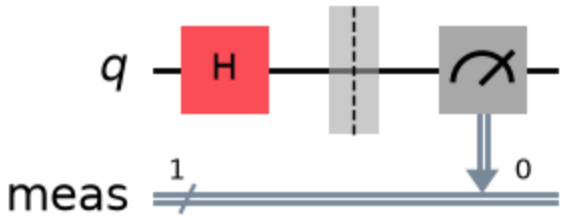



```
{'0': 1024}
```

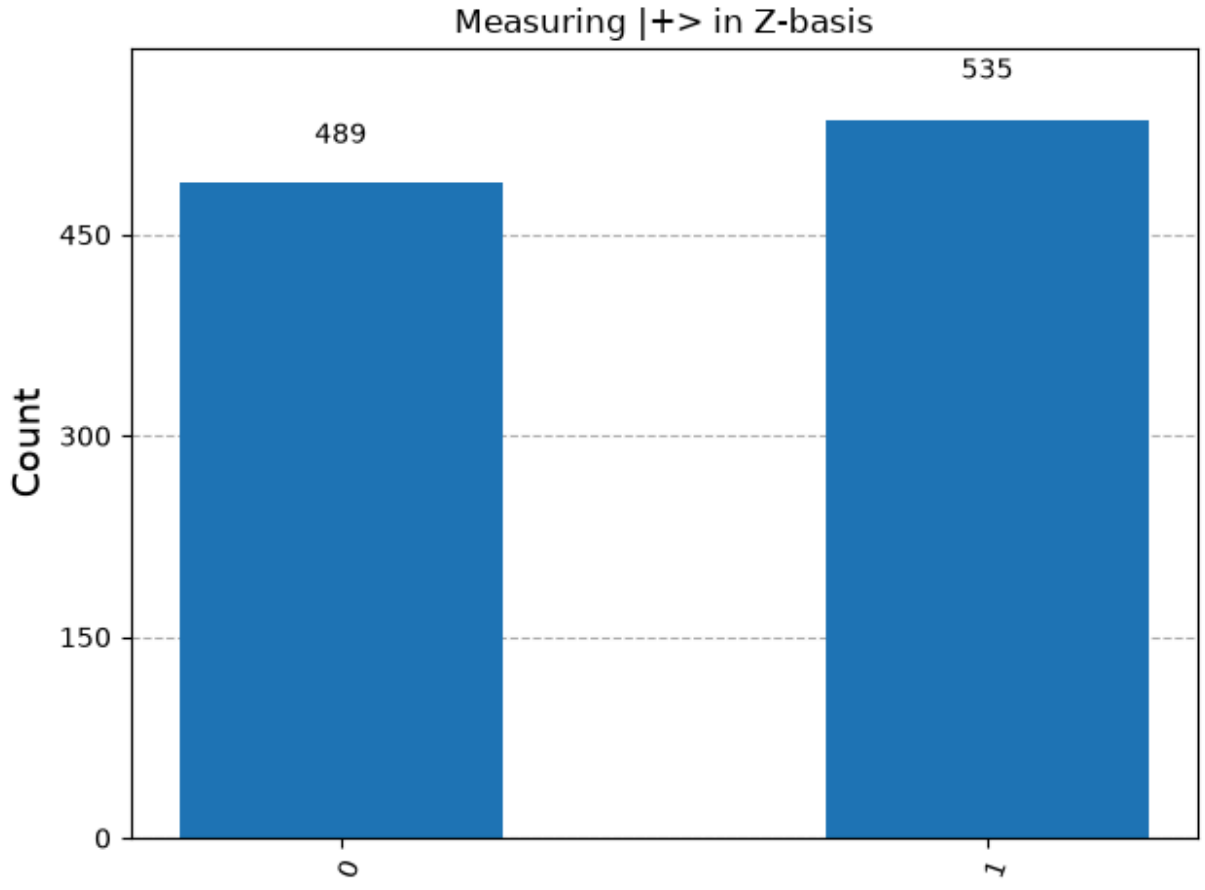
```
# Prepare the |+> State
qc_plus_state = QuantumCircuit(1, name="Prepare |+>")
qc_plus_state.h(0)

# Circuit 1: Measure |+> in the z-basis (should be 50/50)
qc_measure_z = qc_plus_state.copy(name="Measure Z")
```

```
qc_measure_z.measure_all()
display(qc_measure_z.draw("mpl"))
```

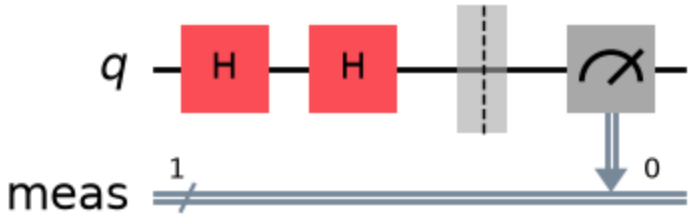



```
run_and_plot(qc_measure_z, title="Measuring |+> in Z-basis")
```

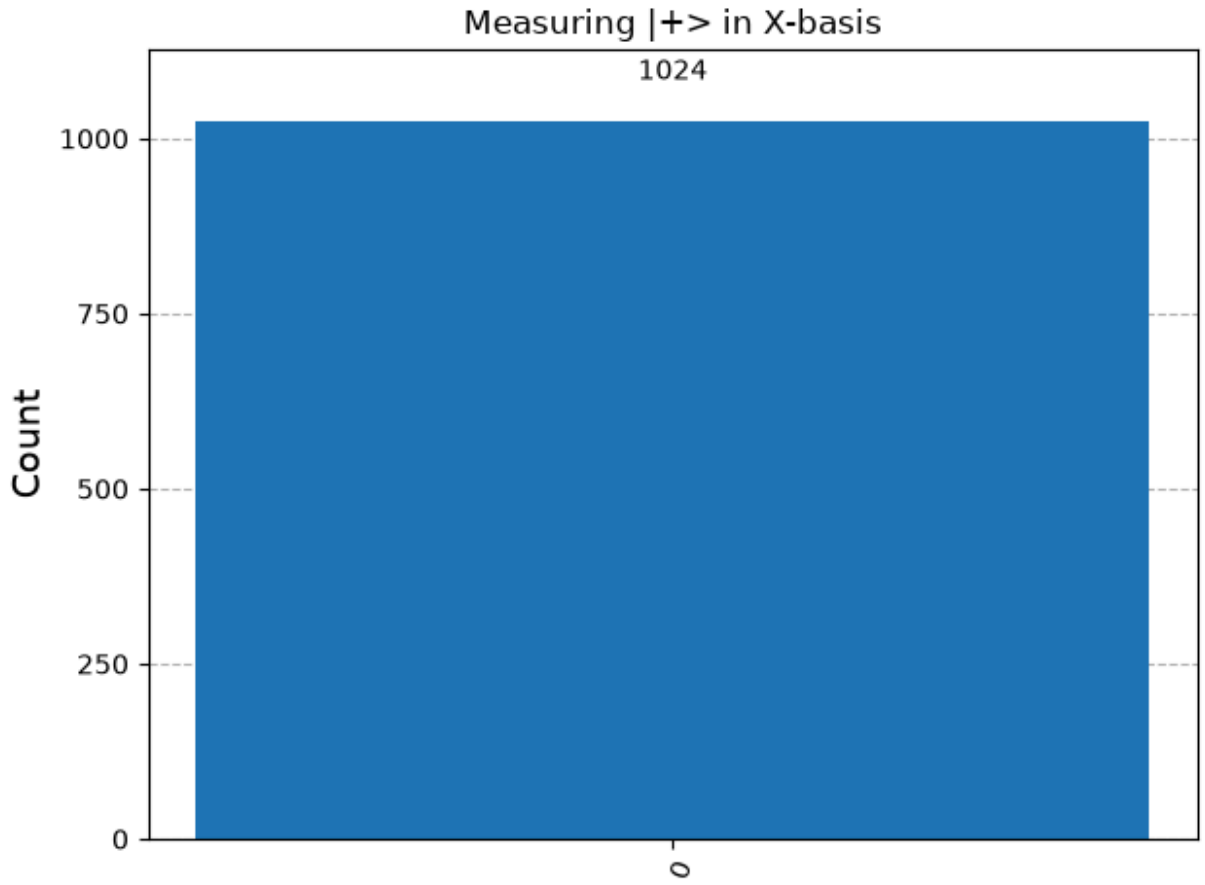



```
{'1': 535, '0': 489}
```

```
# Circuit 2: Measure |+> in the X-basis (should be 100% |+>, so we get "0")
qc_measure_x = qc_plus_state.copy(name="Measure X")
qc_measure_x.h(0)
qc_measure_x.measure_all()
display(qc_measure_x.draw("mpl"))
```




```
run_and_plot(qc_measure_x, title="Measuring |+> in X-basis")
```




```
{'0': 1024}
```

6. SWAP Gate from CNOTs — Gate Decomposition

The SWAP gate swaps the states of two qubits.

It can be "built" from three CNOTs: SWAP(q0,q1) = CX(q0→q1) → CX(q1→q0) → CX(q0→q1)

Why this matters:

- Real quantum computers have a limited gate set.

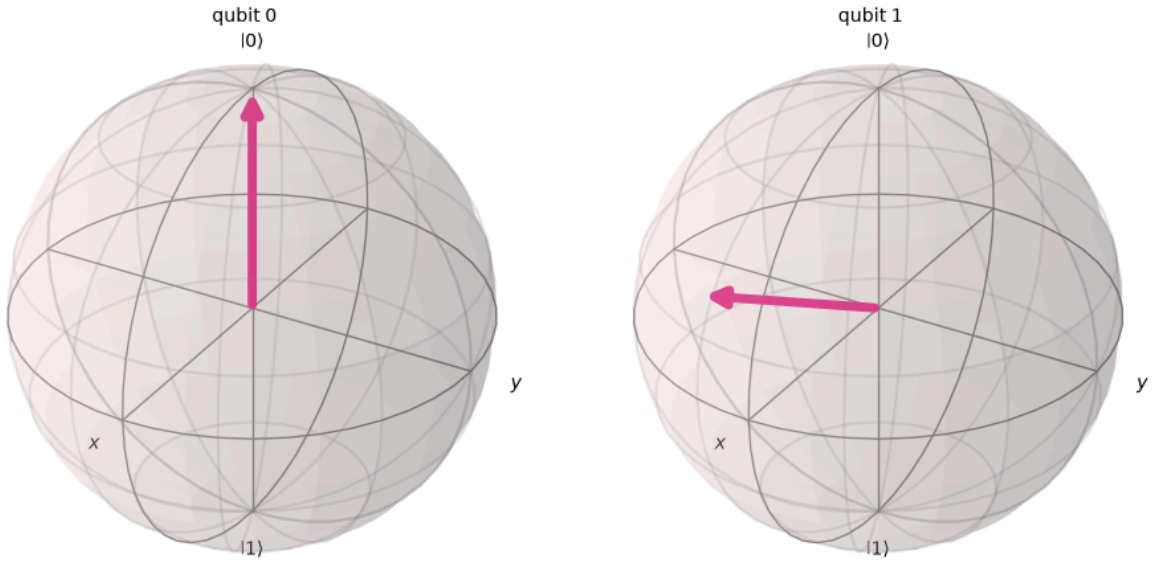
- Complex gates (like SWAP) are decomposed into basic gates.

Here, we:

1. Prepare an arbitrary state on qubit 0.
2. SWAP it to qubit 1.
3. Inspect the statevector and Bloch spheres to confirm.

```
# Build SWAP using three CX gates
qc_swap_nom = QuantumCircuit(2, name="SWAP via 3 CNOTs (no measure)")
qc_swap_nom.ry(pi/3, 0)
qc_swap_nom.rx(pi/5, 0)
qc_swap_nom.cx(0, 1)
qc_swap_nom.cx(1, 0)
qc_swap_nom.cx(0, 1)

state_swap = Statevector.from_instruction(qc_swap_nom)
display(plot_bloch_multivector(state_swap))
print("Statevector:", state_swap)
```

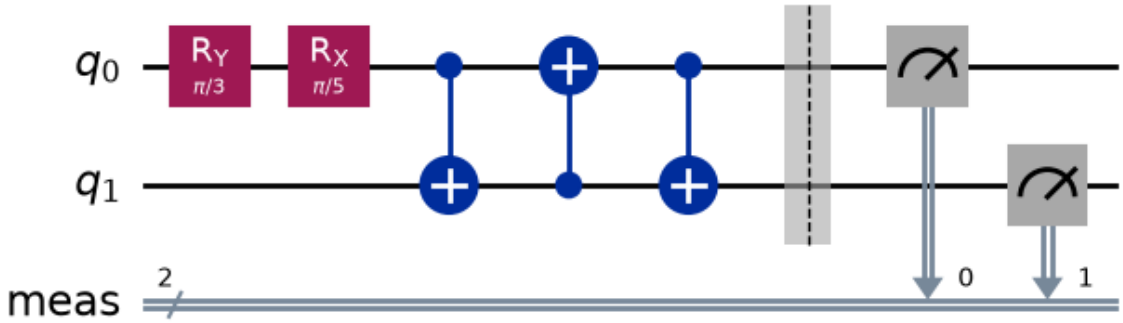



```
Statevector: Statevector([0.8236391 -0.1545085j , 0.        +0.j        ,
             0.47552826-0.26761657j, 0.        +0.j        ],
            dims=(2, 2))
```

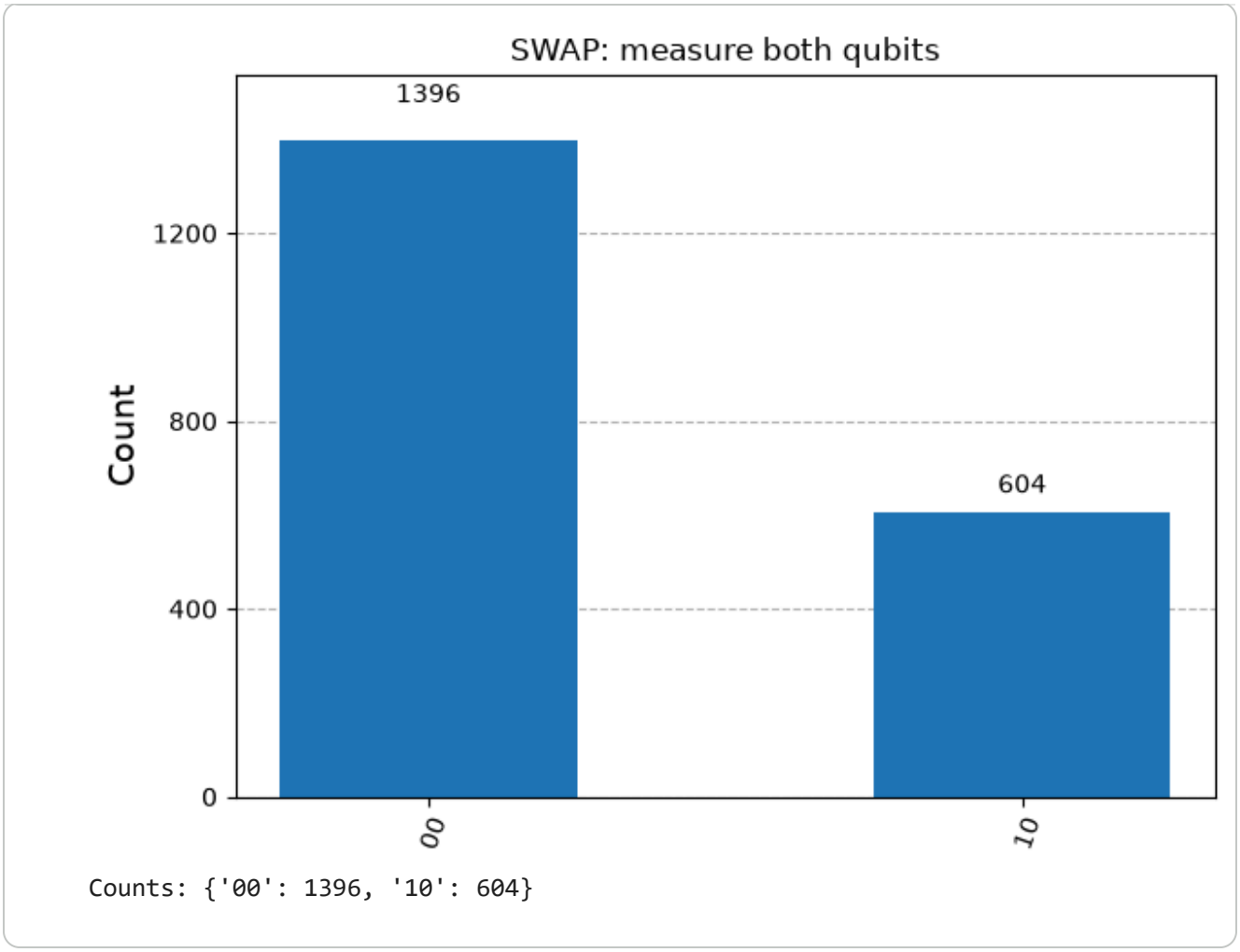
7. SWAP with Measurement

Let’s now add measurement and confirm, via histogram, that the state truly moved to qubit 1.

```
qc_swap = qc_swap_nom.copy()
qc_swap.measure_all()
display(qc_swap.draw("mpl"))
```




```
counts_swap = run_and_plot(qc_swap, shots=2000, title="SWAP: measure both qubit
print("Counts:", counts_swap)
```




```
Counts: {'00': 1396, '10': 604}
```

8. The Bell States: The Four Flavors of Entanglement

We've seen 3-qubit entanglement with the GHZ state. Now, let's explore the simplest, most fundamental examples of entanglement: the four **Bell states**.

These four two-qubit states are maximally entangled and form a complete basis (the "Bell basis"). They are also known as **EPR pairs** (after Einstein, Podolsky, and Rosen) and are the workhorses for algorithms like quantum teleportation and superdense coding.

The four states are:

1. $|\Phi^+\rangle = (|00\rangle + |11\rangle) / \sqrt{2}$ *(Correlated)*
2. $|\Phi^-\rangle = (|00\rangle - |11\rangle) / \sqrt{2}$ *(Correlated, with a phase flip)*
3. $|\Psi^+\rangle = (|01\rangle + |10\rangle) / \sqrt{2}$ *(Anti-correlated)*
4. $|\Psi^-\rangle = (|01\rangle - |10\rangle) / \sqrt{2}$ *(Anti-correlated, with a phase flip)*

Let's build a circuit for each one.

8.1 $|\Phi^+\rangle$: The "Standard" Entangled State

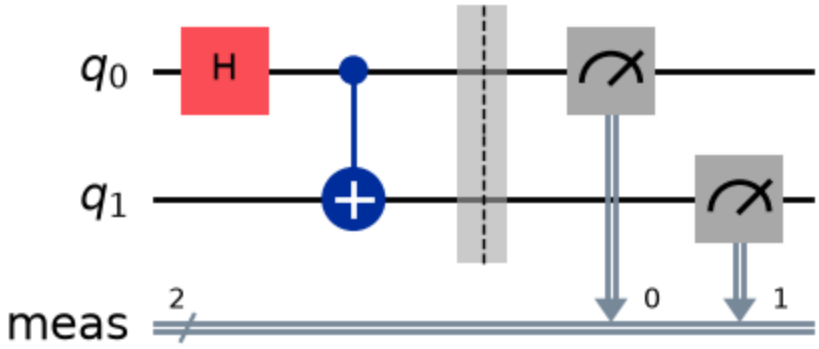
This is the most common Bell state you'll see. It's the two-qubit version of the GHZ state.

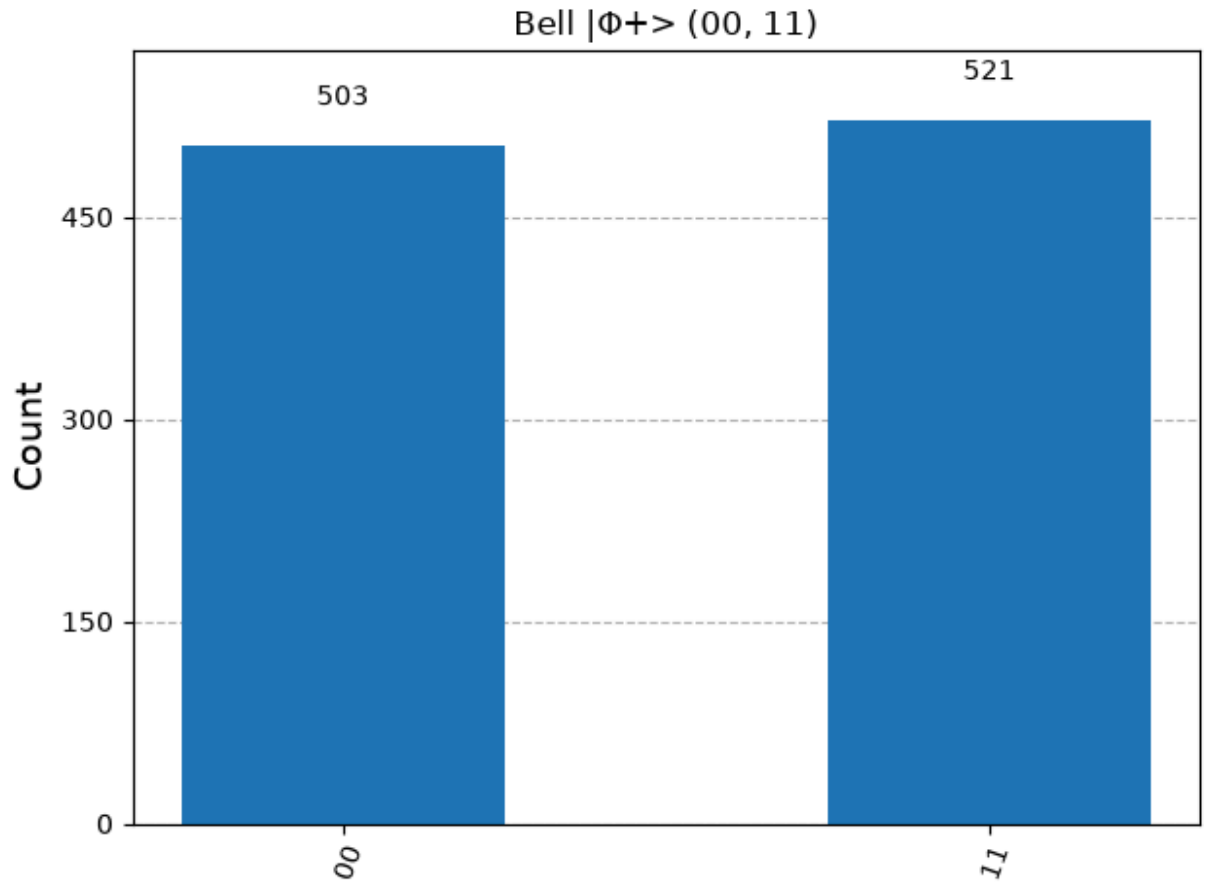
How it works:

1. Apply a Hadamard (`H`) to qubit 0 to create superposition.
2. Apply a CNOT (`CX`) with qubit 0 as control and qubit 1 as target.

Expected Result: The qubits are perfectly correlated. Measurement will yield `00` or `11` with equal probability.

```
qc_phi_p = QuantumCircuit(2, name="Φ+")
qc_phi_p.h(0)
qc_phi_p.cx(0, 1)
qc_phi_p.measure_all()
display(qc_phi_p.draw("mpl"))
run_and_plot(qc_phi_p, title="Bell |Φ+> (00, 11)")
```






```
{'00': 503, '11': 521}
```

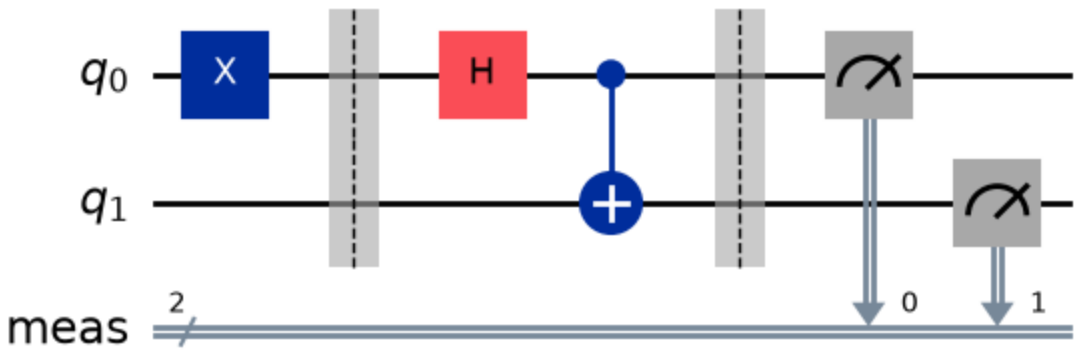
8.2 $|\Phi^-\rangle$: Introducing a Phase Flip

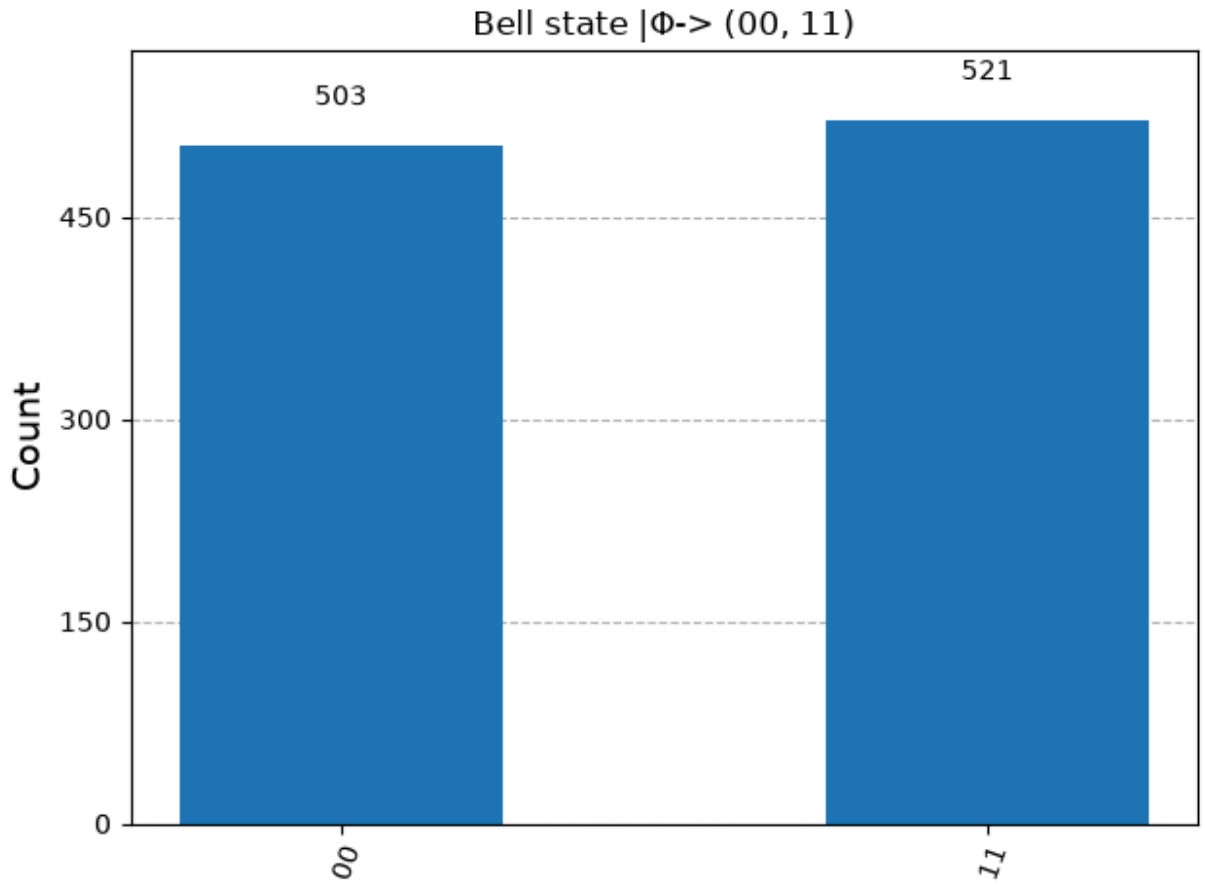
The only difference between $|\Phi^+\rangle$ and $|\Phi^-\rangle$ is the minus sign, which represents a **relative phase difference**.

How it works: To introduce this phase, we start the control qubit in the `|->` state instead of `|+>`. We do this by applying an `X` gate *before* the `H` gate on qubit 0.

Expected Result: Although the quantum state is different, measurement in the Z-basis is "blind" to this overall phase. The outcomes are identical to $|\Phi^+\rangle$: you'll still only see `00` or `11`.

```
# 2. |Φ->
qc_phi_m = QuantumCircuit(2, name="Φ-")
qc_phi_m.x(0)
qc_phi_m.barrier()
qc_phi_m.h(0)
qc_phi_m.cx(0, 1)
qc_phi_m.measure_all()
display(qc_phi_m.draw("mpl"))
run_and_plot(qc_phi_m, title="Bell state |Φ-> (00, 11)")
```






```
{'00': 503, '11': 521}
```

8.3 $|\Psi^+\rangle$: The Anti-Correlated State

In this state, the qubits are **anti-correlated**. If you measure one as `0`, the other is guaranteed to be `1`, and vice versa.

How it works: Instead of starting with the target qubit as `|0>`, we flip it to `|1>` using an `X` gate *before* the CNOT operation.

Expected Result: Measurement will yield `01` or `10` with equal probability.

```
# 3. |Ψ⁺⟩
qc_psi_p = QuantumCircuit(2, name="Ψ+")
qc_psi_p.h(0)
qc_psi_p.x(1)
qc_psi_p.barrier()
qc_psi_p.cx(0,1)
qc_psi_p.measure_all()
display(qc_psi_p.draw("mpl"))
run_and_plot(qc_psi_p, title="Bell state |Ψ⁺⟩ (01, 10)")
```

